

Material complementario clase 4

Dr. Jorge Luis Bazán

16/06/2026

1. Conceptos importantes y Recomendaciones

1.1. Evaluación del desempeño de modelos propuestos

Una de las cuestiones clave para un proyecto exitoso de análisis de datos es ser capaz de evaluar el desempeño de los modelos propuestos.

Las estimaciones confiables del desempeño son de extrema importancia cuando se trata de decidir si implementar los modelos para un uso sistemático. No proporcionar estimaciones confiables puede conducir eventualmente a resultados decepcionantes y comprometer seriamente la aplicación de la minería de datos en una organización.

En esta sección nos concentraremos en la evaluación de un tipo específico de métrica: el *desempeño predictivo*.

Estimar el desempeño predictivo de un modelo, o de un conjunto de modelos, involucra las siguientes cuestiones principales:

- Una tarea predictiva: intentar aproximar una función desconocida $Y = f(x)$, que mapea los valores de un conjunto de predictores en el valor de una variable objetivo. Puede tratarse de un problema de clasificación o de regresión, dependiendo del tipo de Y .
- Un conjunto de datos de entrenamiento

$$\{\langle x_i, y_i \rangle\}_{i=1}^N$$

con valores conocidos de dicho mapeo.

- Uno o más criterios de evaluación del desempeño: métricas de desempeño predictivo, por ejemplo, tasa de error o error cuadrático medio.

En este contexto, nuestro objetivo es decidir cómo obtener estimaciones confiables del desempeño predictivo de las soluciones que consideramos para resolver la tarea usando el conjunto de datos disponible.

La confiabilidad de las estimaciones se relaciona con el problema de la estimación estadística. La cuestión central es si los valores de desempeño predictivo que obtenemos tienen alta probabilidad de ser observados nuevamente cuando aplicamos las mismas soluciones a nuevas muestras de datos.

Calcular el desempeño predictivo de un modelo aplicándolo a los mismos datos usados para obtenerlo no es confiable. El nivel de falta de confiabilidad es proporcional a la capacidad de los modelos para sobreajustar los datos disponibles.

Los modelos con mayor capacidad de ajuste a una muestra dada corren el riesgo de sobreajustarla, fenómeno conocido como *overfitting*. En ese caso, obtendrán una puntuación muy buena si se evalúan sobre esos mismos datos; sin embargo, difícilmente obtendrán el mismo resultado con una muestra diferente.

En este contexto, una regla de oro de la evaluación del desempeño predictivo es:

Siempre evalúe los modelos con datos no vistos.

Esta es la mejor manera de obtener una estimación confiable del error de predicción esperado de un modelo sobre una distribución de datos desconocida.

Un método simple para implementar esta idea consiste en dividir los datos disponibles en dos particiones aleatorias, como se ha hecho al presentar diferentes técnicas de modelamiento:

- (I) un conjunto de entrenamiento usado para desarrollar los modelos;
- (II) un conjunto de prueba para calcular su desempeño predictivo.

Sin embargo, debemos intentar garantizar que esta división aleatoria se repita varias veces para aumentar la confiabilidad estadística del proceso de estimación. Con varias repeticiones se obtiene un conjunto de puntajes y luego se puede calcular la media para obtener un error de predicción promedio muestral, que puede interpretarse como una estimación del verdadero error de predicción poblacional.

Además, podemos complementar esta estimación con su respectivo error estándar. Una metodología experimental debería permitirnos obtener varios puntajes de error de predicción, que luego podemos usar para calcular el error de predicción promedio de la muestra:

$$\bar{E} = \frac{1}{k} \sum_{i=1}^k E_i$$

El respectivo error estándar de esta estimación está dado por:

$$EP(\bar{E}) = \frac{s_E}{\sqrt{k}}$$

donde s_E es la desviación estándar muestral de $\bar{E}$, medida como:

$$s_E = \sqrt{\frac{1}{k-1} \sum_{i=1}^k (E_i - \bar{E})^2}$$

1.2. Paquetes en R y Python

1.2.1. Paquetes en R

En R existen varios paquetes que facilitan la realización de experimentos orientados a estimar y comparar el desempeño predictivo de diferentes modelos. Algunos ejemplos son el paquete **caret** (Kuhn, 2016), el paquete **mlr** (Bischl et al., 2016) y el paquete **performanceEstimation** (Torgo, 2014b). Este último proporciona una infraestructura muy general para este tipo de experimentos.

Kuhn, M. (2016). *caret: Classification and Regression Training*. R package version 6.0-68.

- <https://topepo.github.io/caret/>
- <https://cran.r-project.org/web/packages/caret/index.html>

Bischl, B., Lang, M., Richter, J., Bossek, J., Judt, L., Kuehn, T., Studerus, E., Kotthoff, L., and Julia, S. (2016). *mlr: Machine Learning in R*. R package version 2.8.

- <https://mlr.mlr-org.com/>
- <https://mlr3.mlr-org.com/>
- <https://cran.r-project.org/web/packages/mlr/index.html>
- <https://cran.r-project.org/web/packages/mlr3/index.html>

Torgo, L. (2014b). An infrastructure for performance estimation and experimental comparison of predictive models in R. *CoRR*, abs/1412.0436.

- <https://github.com/ltorgo/performanceEstimation>
- <https://cran.r-project.org/web/packages/performanceEstimation/index.html>

1.2.2. Paquetes en Python

En Python existen varios paquetes que facilitan la realización de experimentos destinados a estimar y comparar el desempeño predictivo de diferentes modelos.

Entre los ejemplos se incluyen `pandas` y `scikit-learn`. Este último proporciona una infraestructura muy general para este tipo de experimentos.

- [Data Resampling using Python. Data resampling is a technique that... | by Abel AK | Medium](#)
- [PyCaret — pycaret 3.0.4 documentation](#)

También podemos mencionar `PyCaret`, una biblioteca de aprendizaje automático de código abierto y bajo código en Python. Funciona de manera automática con flujos de aprendizaje de máquina y ofrece herramientas de extremo a extremo para experimentación y gestión de modelos. Su objetivo es acelerar la experimentación y producir resultados de manera eficiente.

1.3. Procedimientos recomendados

Según Torgo (2010), se recomiendan los siguientes procedimientos:

- **Conjuntos de datos pequeños, con menos de algunas centenas de observaciones.** Usar entre 100 y 200 repeticiones de un procedimiento de *bootstrap*.
- **Conjuntos de datos de tamaño medio, con menos de algunos miles de casos.** Usar algunas repeticiones de validación cruzada de 10 particiones, o *10-fold cross-validation*.
- **Conjuntos de datos grandes.** Usar *holdout*, por ejemplo una división 70 %-30 %, o algunas repeticiones de submuestreo aleatorio.

Aunque existen otras metodologías alternativas a las presentadas aquí, los esquemas descritos son los más razonables para usar. Naturalmente, algunas de estas decisiones dependen de:

- a) el poder de cómputo disponible para realizar estos experimentos;
- b) el número de tareas y/o flujos de trabajo.

Torgo, L. (2010). *Data Mining with R: Learning with Case Studies*. Chapman and Hall/CRC Press. ISBN 9781439810187.

En este contexto, también es importante tener presente que varias funciones de R incluyen opciones de computación paralela, lo que puede mejorar la eficiencia de los flujos de trabajo. Un ejemplo es la función `performanceEstimation()`, que puede aprovechar distintos *backends* de computación paralela: desde la paralelización local simple usando múltiples núcleos de procesadores multicore, hasta configuraciones más sofisticadas con clústeres de computadores.

En términos de análisis de resultados, a veces es importante verificar la significancia estadística de las diferencias observadas entre el desempeño de flujos de trabajo alternativos considerados para resolver una o más tareas. Demsar (2006) proporciona un estudio ampliamente reconocido sobre este tema. En la descripción siguiente seguimos principalmente sus recomendaciones.

Distinguiremos entre dos configuraciones básicas:

- (I) comparar varios flujos de trabajo en una única tarea predictiva;
- (II) compararlos en varias tareas.

Para el primer caso, el procedimiento aceptado consiste en usar la prueba estadística de rangos con signo de Wilcoxon para comparar los puntajes de los diferentes flujos de trabajo. Esta es una prueba de hipótesis estadística no paramétrica que puede utilizarse para realizar comparaciones pareadas de dos rangos medios poblacionales.

La prueba de Wilcoxon es preferible a la prueba *t* de Student pareada, también usada en este tipo de comparación experimental, porque la mayoría de los métodos experimentales no satisfacen algunos de los supuestos de esta última prueba. En este contexto, podemos usar la prueba de rangos con signo de Wilcoxon para verificar la significancia estadística de las diferencias entre el desempeño de cualquier par de flujos de trabajo.

Para la segunda configuración, donde se comparan varios flujos de trabajo sobre un conjunto de tareas predictivas, las recomendaciones de Demsar (2006) indican proceder en dos etapas. Primero se usa una prueba de Friedman para verificar la validez de la hipótesis nula de que todos los flujos de trabajo son equivalentes y, por lo tanto, sus clasificaciones en las tareas son iguales.

Si no podemos rechazar esta hipótesis con el nivel de confianza seleccionado, el análisis se detiene y se concluye que no hay evidencia suficiente para rechazar la hipótesis de equivalencia entre los flujos de trabajo. En caso contrario, se puede continuar con la segunda etapa.

Si estamos interesados en todas las comparaciones pareadas entre los flujos de trabajo, debemos usar una prueba post-hoc de Nemenyi para verificar la hipótesis nula de que no existe diferencia significativa entre los rangos de un determinado par de flujos de trabajo.

Si, en cambio, estamos interesados en todas las comparaciones pareadas con un flujo de trabajo de referencia, debemos usar la prueba post-hoc de Bonferroni-Dunn para verificar la hipótesis nula de que no existe diferencia significativa entre los rangos de un flujo de trabajo determinado y la línea base.

Demsar, J. (2006). Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, 7:1–30.

2. Otras informaciones

2.1. Otras métricas para datos desbalanceados

Cuando tenemos pocos 0s o 1s en dato binarios, algunas métricas pueden ser cuestionables. Se recomienda consultar las siguientes citas

Bazán, J. (2023) Binary classification and evaluation metrics using supervised machine learning models to imbalanced data. Workshop on Machine and Statistical Learning. Universidad Adolfo Ibañez. June 14. 2023. Available in https://ml4brainhealth.uai.cl/wp-content/uploads/2023/06/PPT-Workshop_Jorge-Bazan.pdf

de la Cruz Huayanay, A., Bazán, J.L. & Russo, C.M. Performance of evaluation metrics for classification in imbalanced data. Comput Stat 40, 1447–1473 (2025). <https://doi.org/10.1007/s00180-024-01539-5>

y los códigos disponibles en https://github.com/aldehu/evaluation_metrics que implementan una estimación bayesiana de modelos de regresión con enlaces asimétricos en Python y métricas en el contexto de datos desbalanceados.

2.2. Aspectos éticos

Se recomienda revisar

Tabares Reiné (2023) Aspectos transparentes, éticos y responsables en Inteligencia Artificial. Workshop on Machine and Statistical Learning. Universidad Adolfo Ibañez. 14 de Junho 2023. Available in https://ml4brainhealth.uai.cl/wp-content/uploads/2023/06/PPT-Workshop_Reinél-Tabares.pdf